

GROUP 06 MAPD-B 16.09.2026

Streaming processing of the QUAX experiment data for the detection of galactic axions



Erario Francesco, Lorenzetto Andrea, Mazzilli Laura, Zuccotti Nicola

Index

- Introduction and Kaska infrastructure

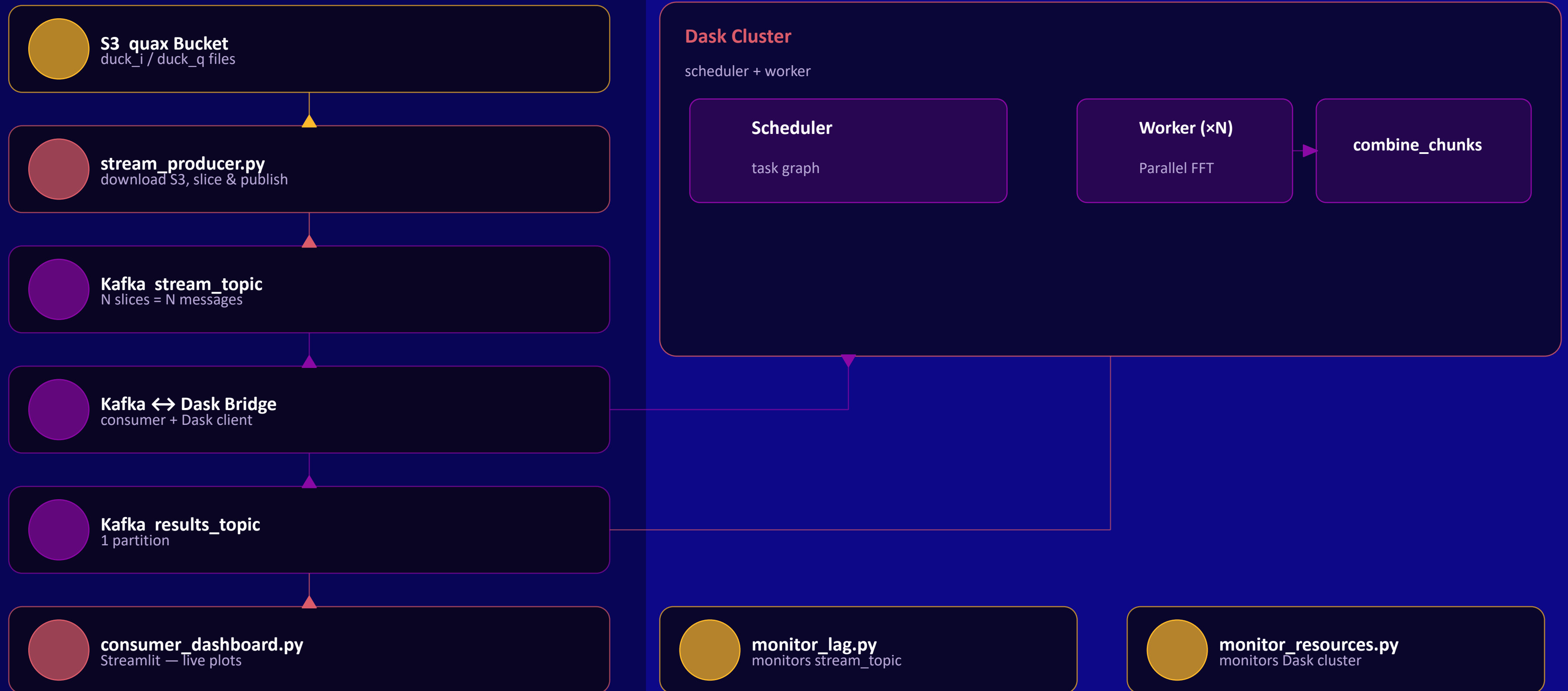
- Producer

- Dask

- Dashboard

Streaming Data Pipeline Architecture

Real-time FFT processing with Kafka & Dask





INTRODUCTION:

V.M. schema

VM 1: ACCESS

3 KAFKA BROKER:

VM 2

VM 3

VM 4

DASK CLUSTER

VM 5: SCHEDULER

4 WORKER:

VM 6

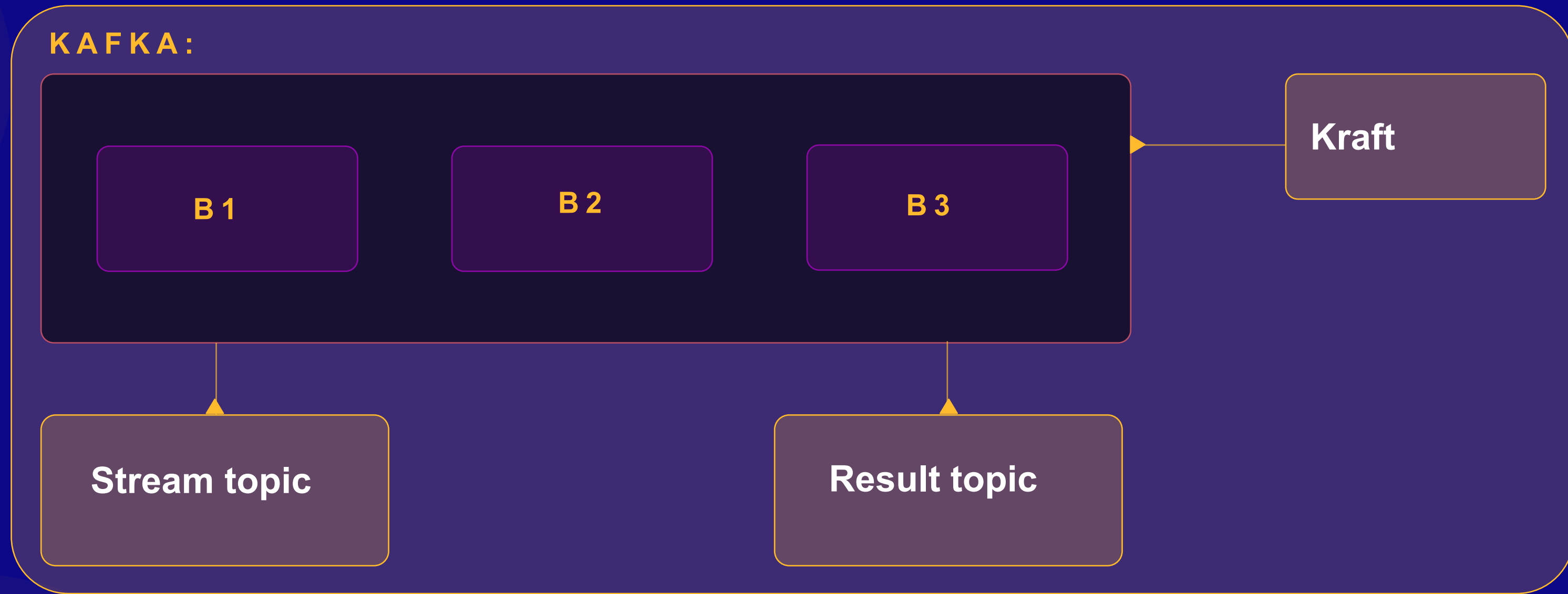
VM 7

VM 8

VM 9

DASK
S.N.

VM 10



```
def create_topic(admin, name, partitions, replication_factor):  
    topic = NewTopic(name=name, num_partitions=partitions, replication_factor=replication_factor)
```

```
admin = KafkaAdminClient(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS)
```

```
create_topic(admin, STREAM_TOPIC, args.partitions, args.replication_factor)  
create_topic(admin, RESULTS_TOPIC, 1, args.replication_factor)
```

CONSUMER GROUP:

- 1/2 consumers depending on the run
- start from the “earliest” available offset
- kafka periodic automatic commit

```
# create the data stream consumer
consumer = KafkaConsumer(
    STREAM_TOPIC,
    bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
    group_id=args.group_id,
    auto_offset_reset="earliest",
    max_partition_fetch_bytes=MAX_BYTES,
    fetch_max_bytes=MAX_BYTES,
    consumer_timeout_ms=args.consumer_timeout_ms,
)
```

MESSAGE READING:

Producer sends message using a key

same batch = same partition

```
pending = {}

for message in consumer:
    print(f"Got offset={message.offset} size={len(message.value)}", flush=True)
    raw = message.value # select raw bytes

    header_len = struct.unpack(">I", raw[:4])[0]
    header_bytes = raw[4:4 + header_len]
    header = json.loads(header_bytes)

    batch_id = header["batch_id"]
    slice_idx = header["slice_idx"]
    n_slices = header["n_slices"]
    timestamp = header["timestamp"]

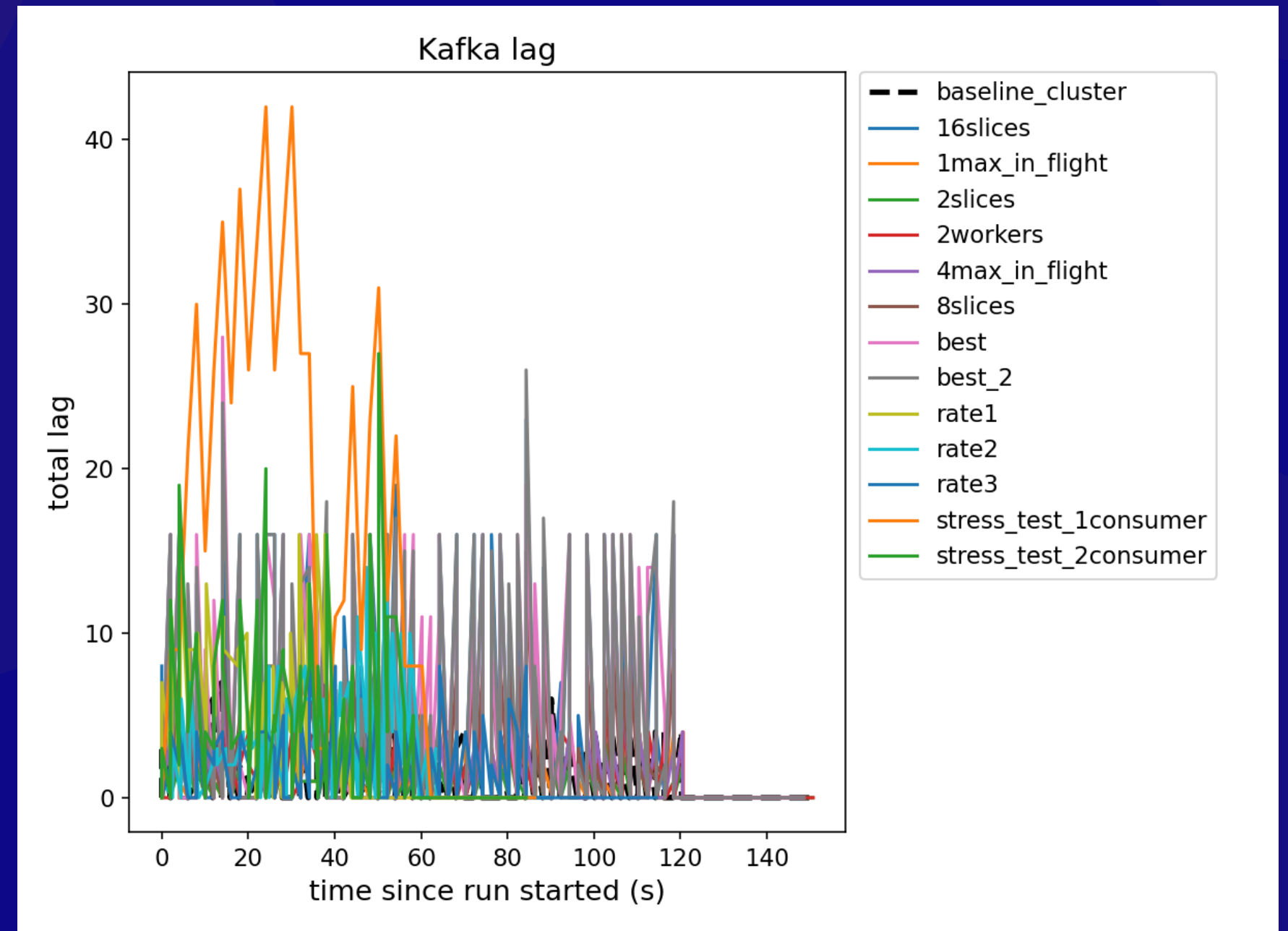
    if batch_id not in pending:
        pending[batch_id] = []

    with bench.timed(batch_id, "dask_submit"):
        raw_future = client.scatter(raw)
        chunk_future = client.submit(compute_fft_chunk, raw_future,
                                     header_len,
                                     header["n_bytes_i"], header["n_bytes_q"])

    pending[batch_id].append(chunk_future)
```

CONSUMER LAG AND BENCHMARKS:

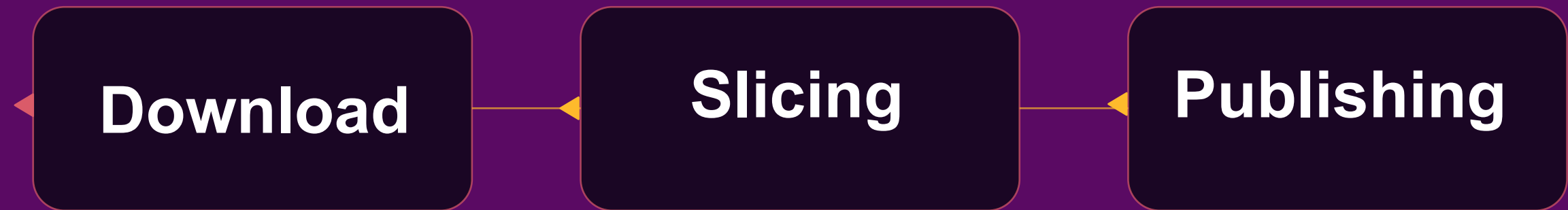
- A monitor script tracks the consumer lag
- We run the pipeline changing one parameter at each run



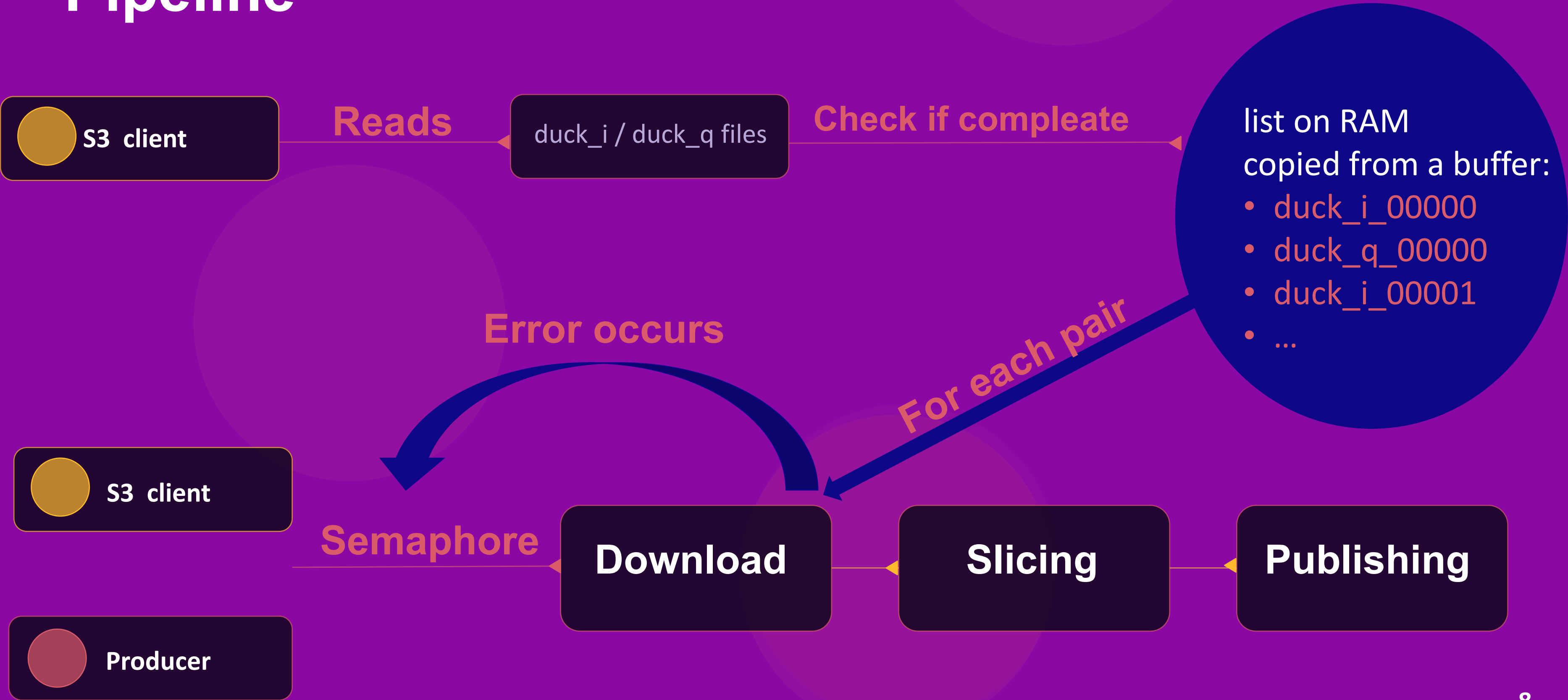
$$\text{Lag} = \text{end_offset} - \text{committed_offset}$$



PRODUCER

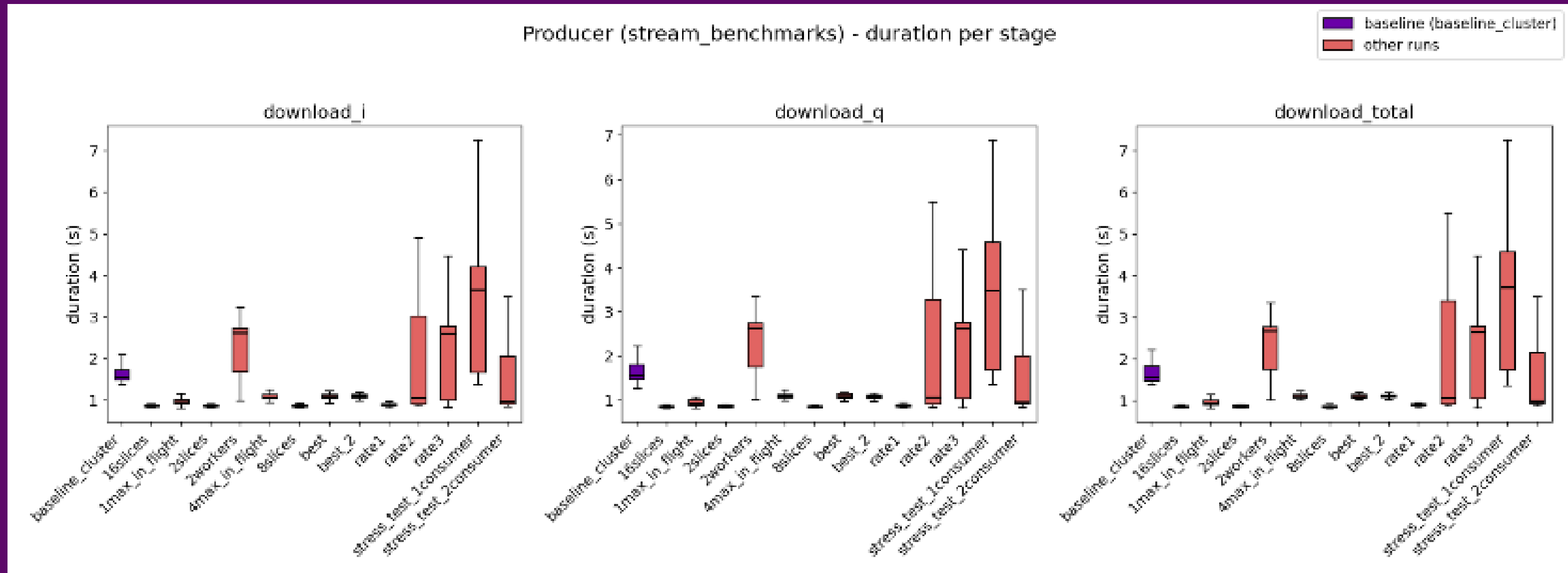


Pipeline



DOWNLOAD'S PARALLELIZATION:

- 32 Mb per file / 16 threads per file
- GIL released during I/O calls + orthogonal threads in boto3 + buffer on RAM



ERROR MANAGEMENT:

Permanent errors:

- **Boto3 - No file, no bucket, 404**
- **Kafka - Message size**

Transient errors:

- **No connection**
- **Bad connection**
- **ReadTimeoutError**
- **...**

Retry with exponential time delay with a max number of retries only for transient errors

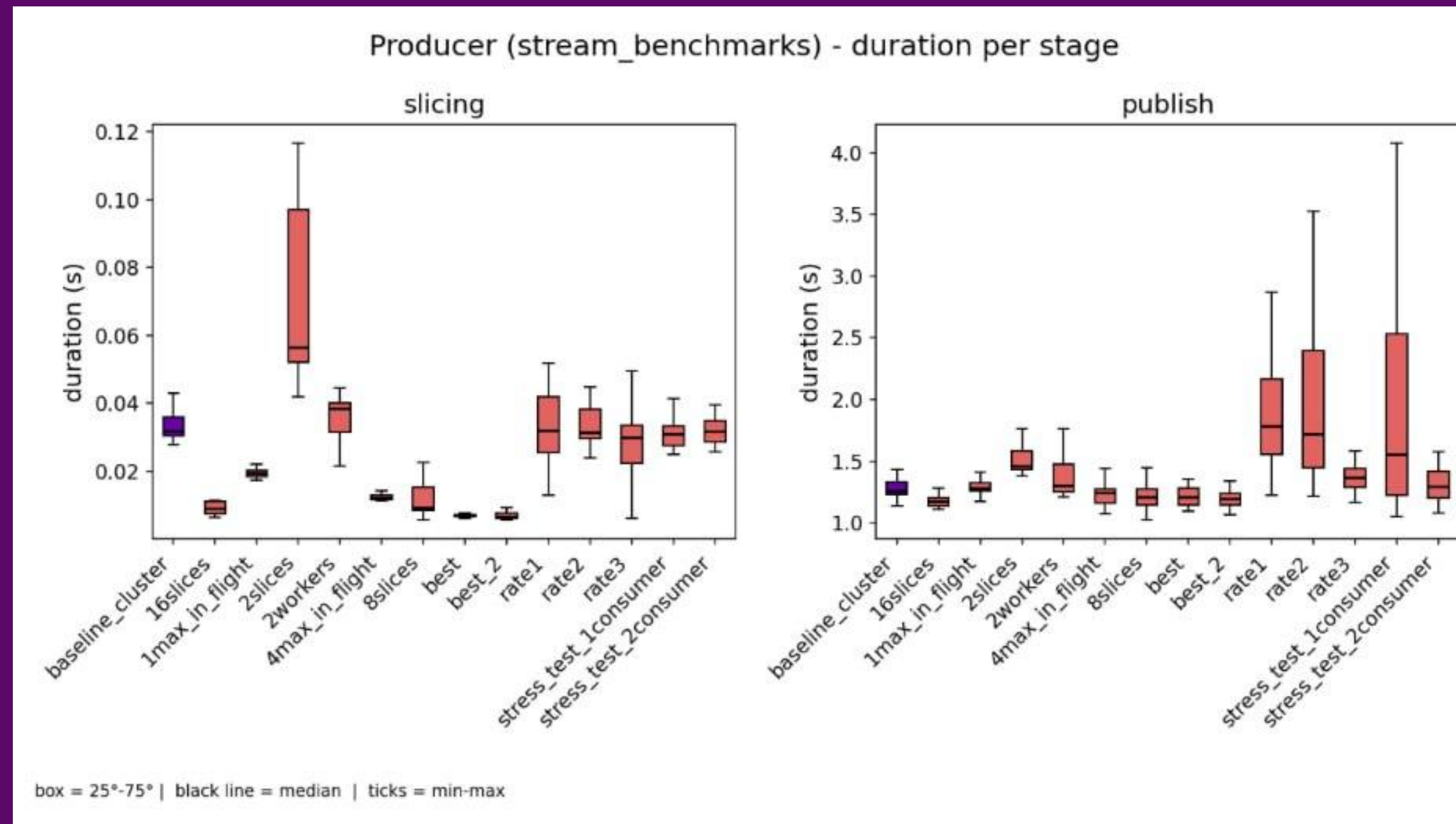
MESSAGE:

**Number of slices
proportional to 2.**

- **Lenght header: 4 bytes big endian unsigned integer**
- **Json header: batch ID, slice ID, # slices, # bytes i, # bytes q, type, timestamp**
- **Concatenation of the slices: first i then the q coupled**

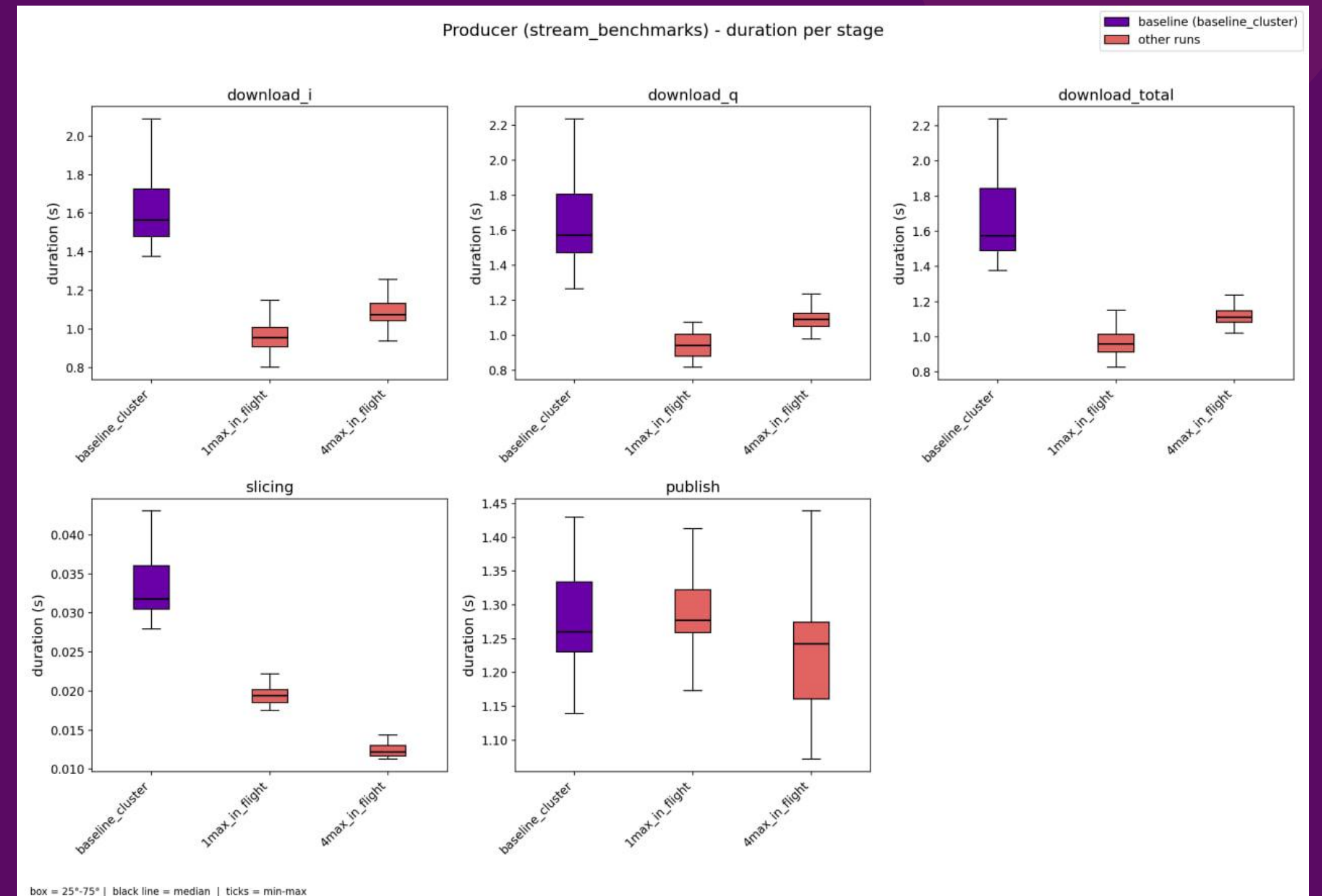
PUBLISH:

- **Lazy:** all calls in a batch are sent one after another, and only in the end we collect the futures
- **acks = “all”** for max durability



SEMAPHORE

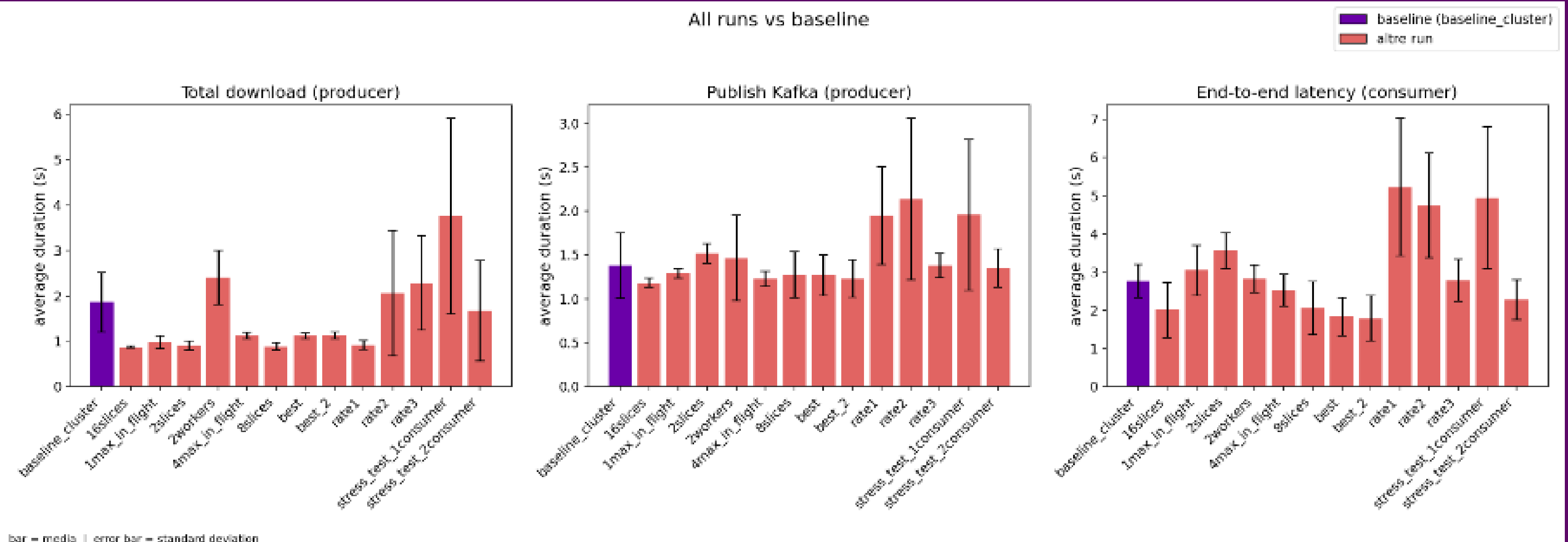
- Maximum number on batch
- No priority



BENCHMARKS

- timestamp - batch ID - stage - duration - # slices - run ID
- Class called thanks to a contextmanager
- Runs organized through repos and csv files via command line

All runs vs baseline



DASK CLUSTER

VM 5: SCHEDULER

4 WORKER:

VM 7

VM 6

VM 8

VM 9

DASK
S.N.

VM 10

FFT COMPUTATION:

DASK

```

# unpack the raw message and compute fft for chunks of data
def compute_fft_chunk(raw, header_len, n_bytes_i, n_bytes_q):
    # extract payload
    payload = memoryview(raw)[4 + header_len:]
    # extract slice i
    # data are float32 little-endian -> convert using dtype="<f4"
    # frombuffer gives a 1D flat array
    chunk_i = np.frombuffer(payload[:n_bytes_i], dtype="<f4")
    # extract slice q
    chunk_q = np.frombuffer(payload[n_bytes_i:n_bytes_i + n_bytes_q], dtype="<f4")

    signal = chunk_i + 1j * chunk_q
    matrix = signal.reshape(-1, FFT_SIZE)
    n_scans = len(matrix)
    ps = np.abs(np.fft.fft(matrix, axis=1)) ** 2
    return ps.mean(axis=0), ps.std(axis=0), n_scans

```

```

# combine results from fft
def combine_chunks(batch_id, timestamp, results):
    total_scans = 0
    for r in results:
        total_scans += r[2]
    #mean
    weighted_mean_sum = 0
    for r in results:
        weighted_mean_sum += r[2] * r[0]
    combined_mean = weighted_mean_sum / total_scans
    #comb var
    weighted_var_sum = 0
    for r in results:
        weighted_var_sum += r[2] * (r[1] ** 2 + (r[0] - combined_mean) ** 2)
    combined_std = np.sqrt(weighted_var_sum / total_scans)

    return batch_id, timestamp, combined_mean, combined_std

```

THE FFT FUNCTIONS

The two functions to do the computation

THE BASIC SCENARIO:

Offline single node

```
for raw, header_len, n_bytes_i, n_bytes_q in messages:
    t_scatter_start = time.perf_counter()
    1 raw_future = client.scatter(raw)
    t_scatter += time.perf_counter() - t_scatter_start

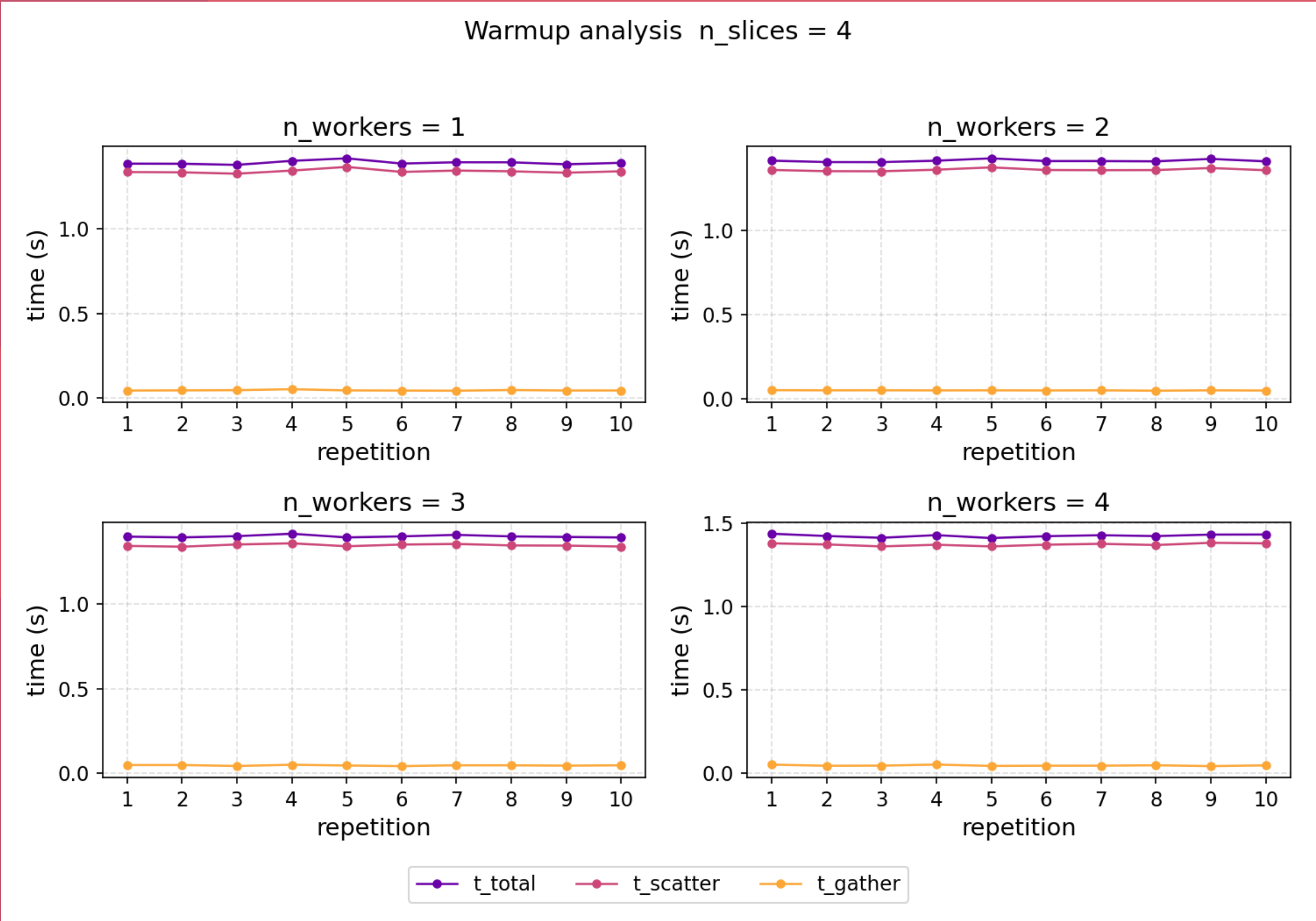
    t_submit_start = time.perf_counter()
    2 chunk_future = client.submit(compute_fft_chunk, raw_future, header_len, n_bytes_i, n_bytes_q)
    t_submit += time.perf_counter() - t_submit_start

    chunk_futures.append(chunk_future)

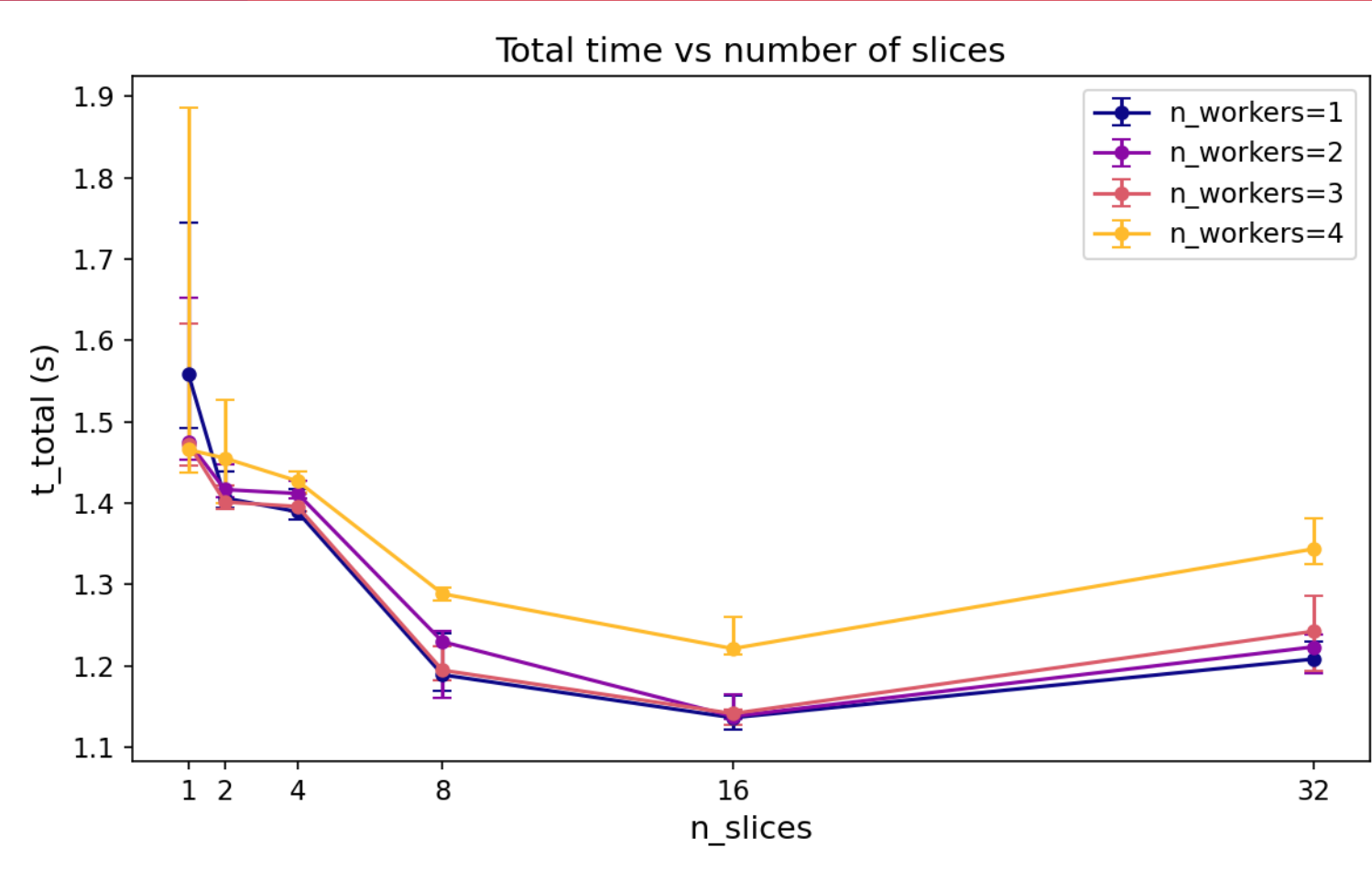
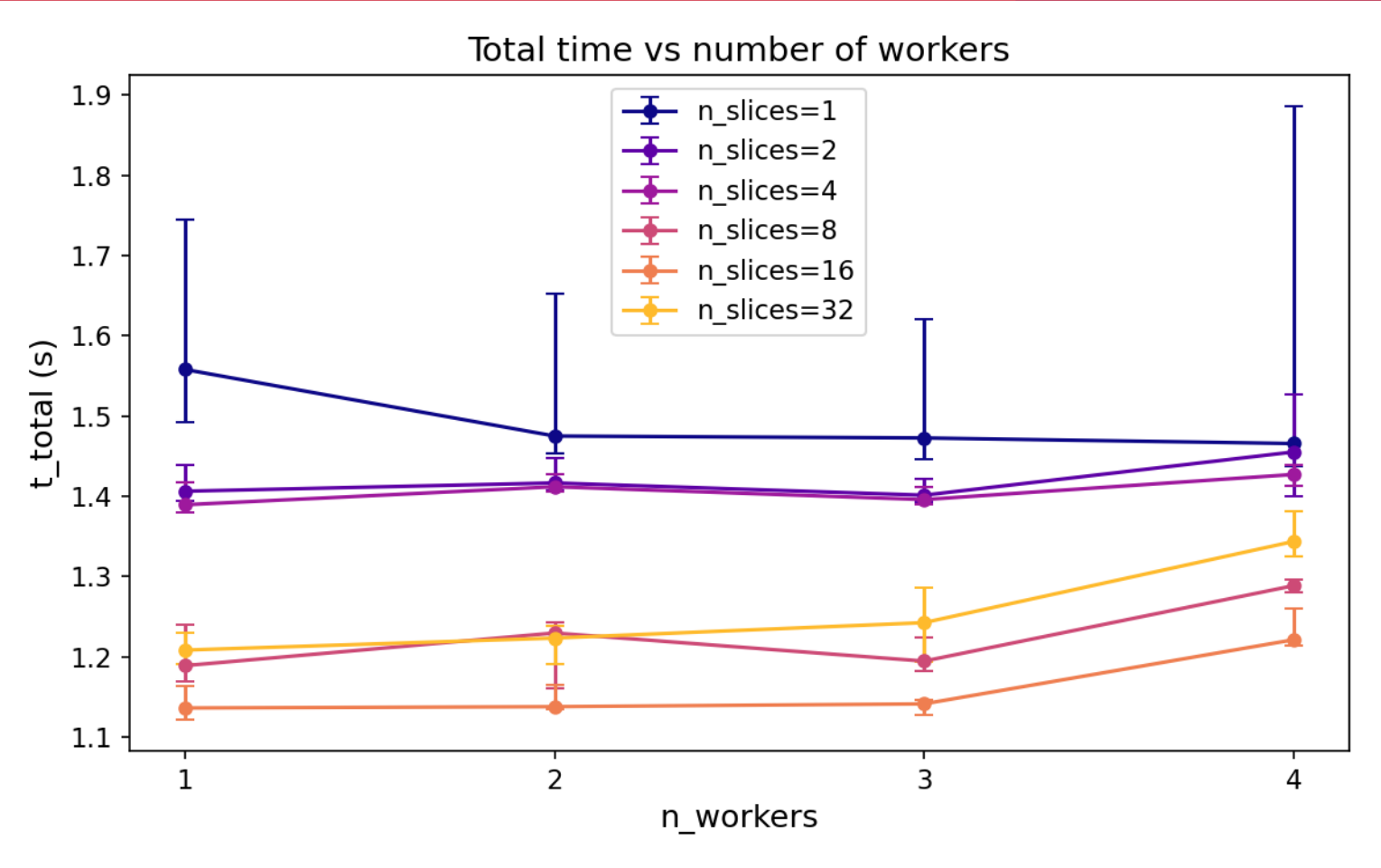
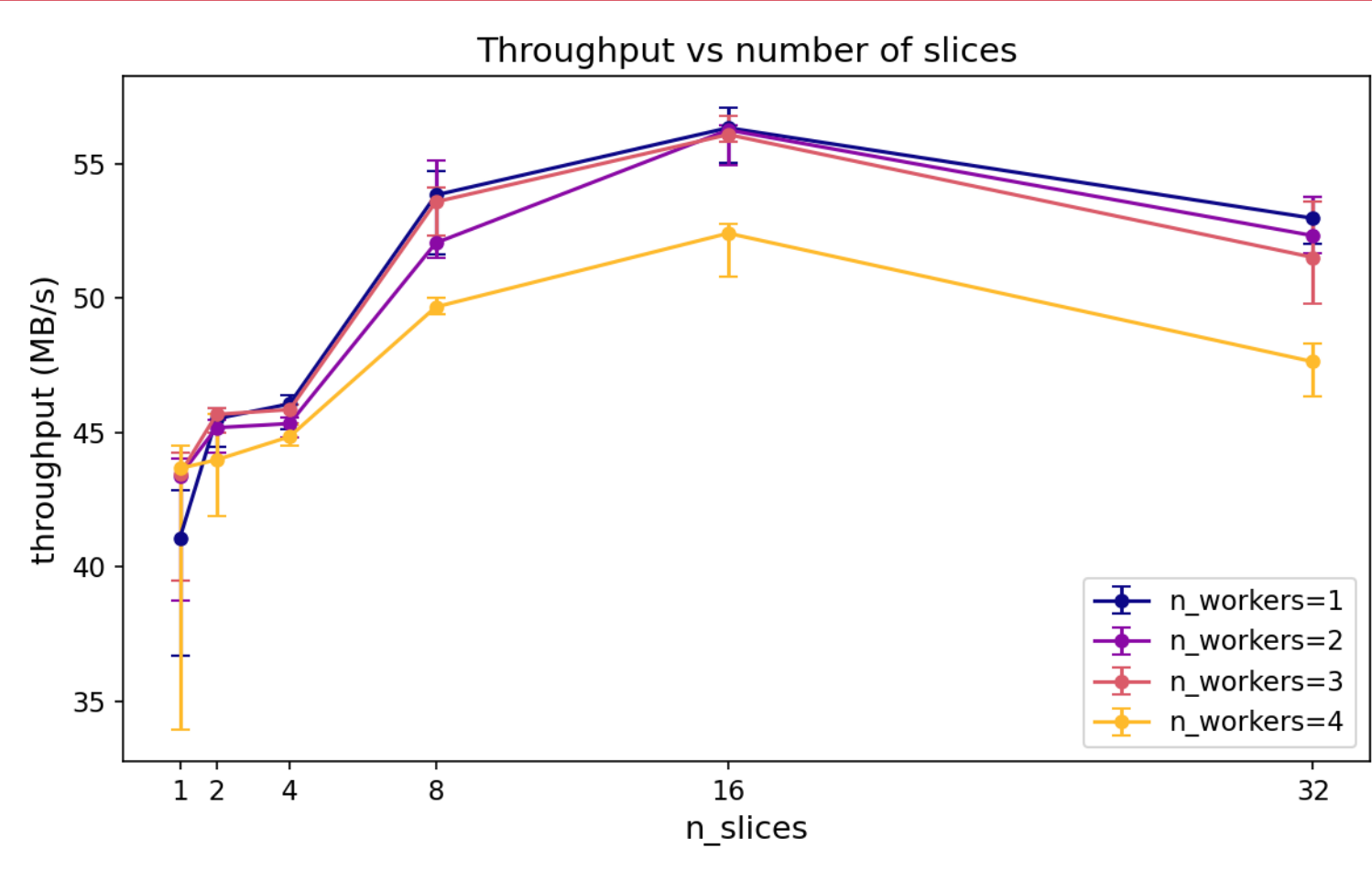
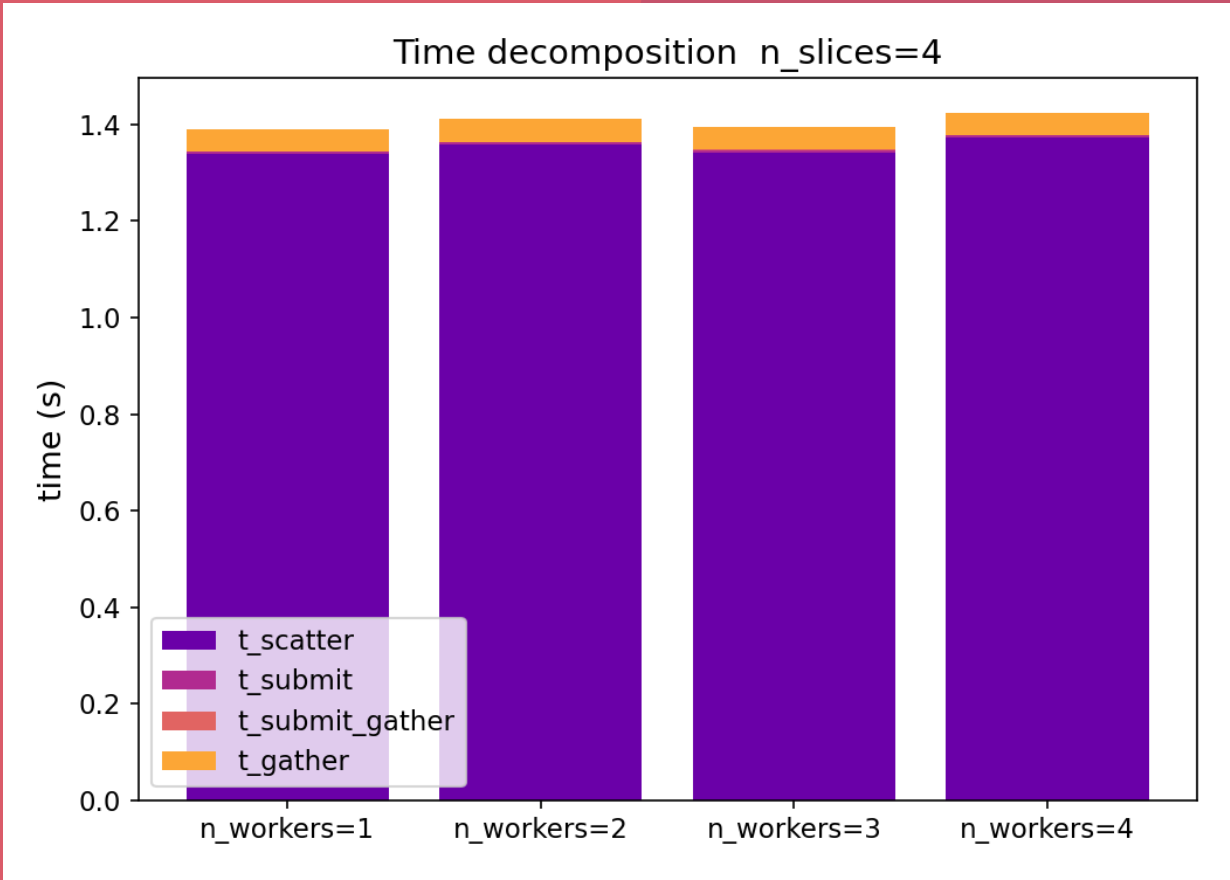
t_submit_gather_start = time.perf_counter()
combine_future = client.submit(combine_chunks, batch_id, timestamp, chunk_futures) 3
t_submit_gather = time.perf_counter() - t_submit_gather_start

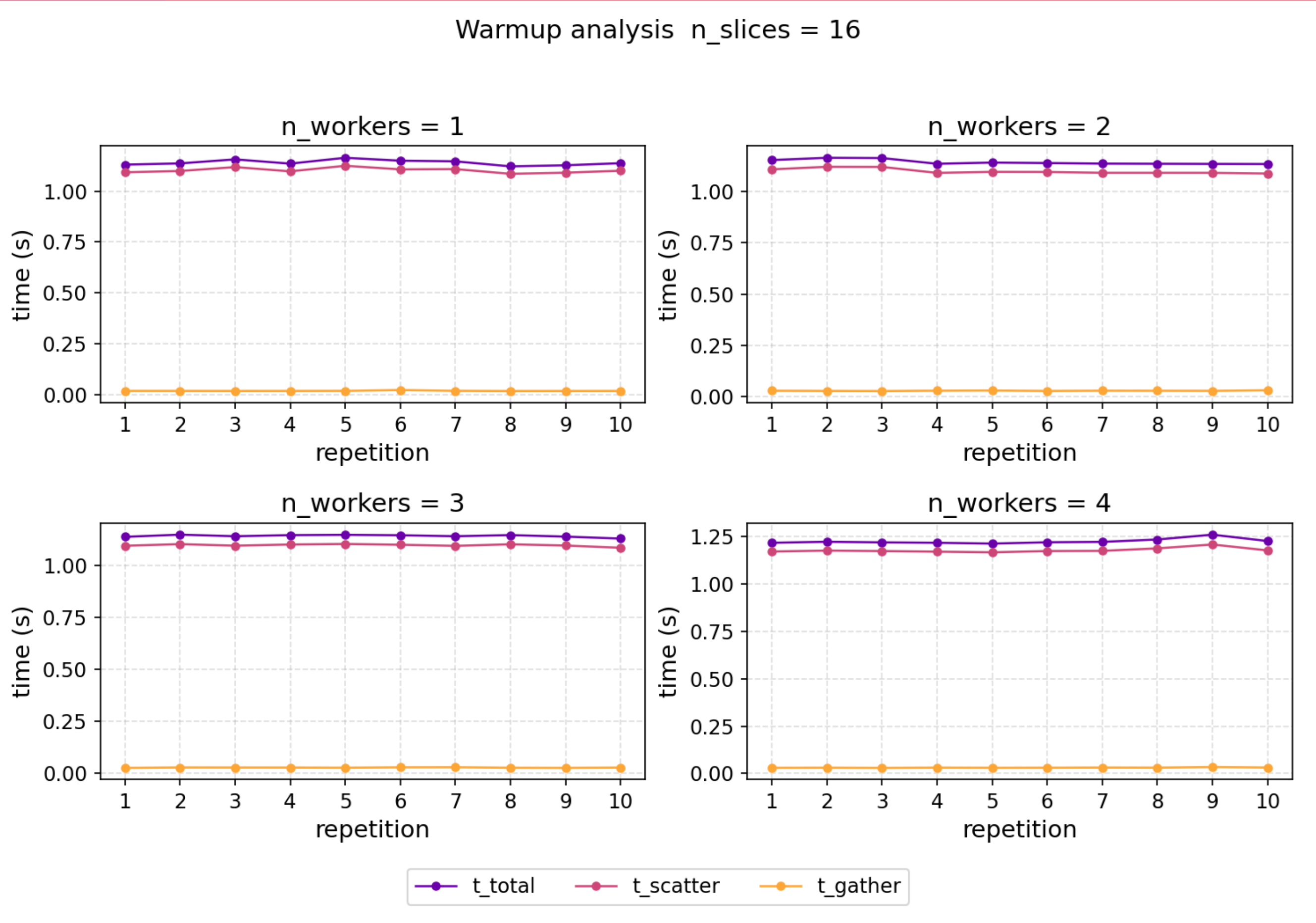
t_gather_start = time.perf_counter()
_, _, combined_mean, combined_std = client.gather(combine_future) 4
t_gather = time.perf_counter() - t_gather_start

t_total = time.perf_counter() - t_total_start
```

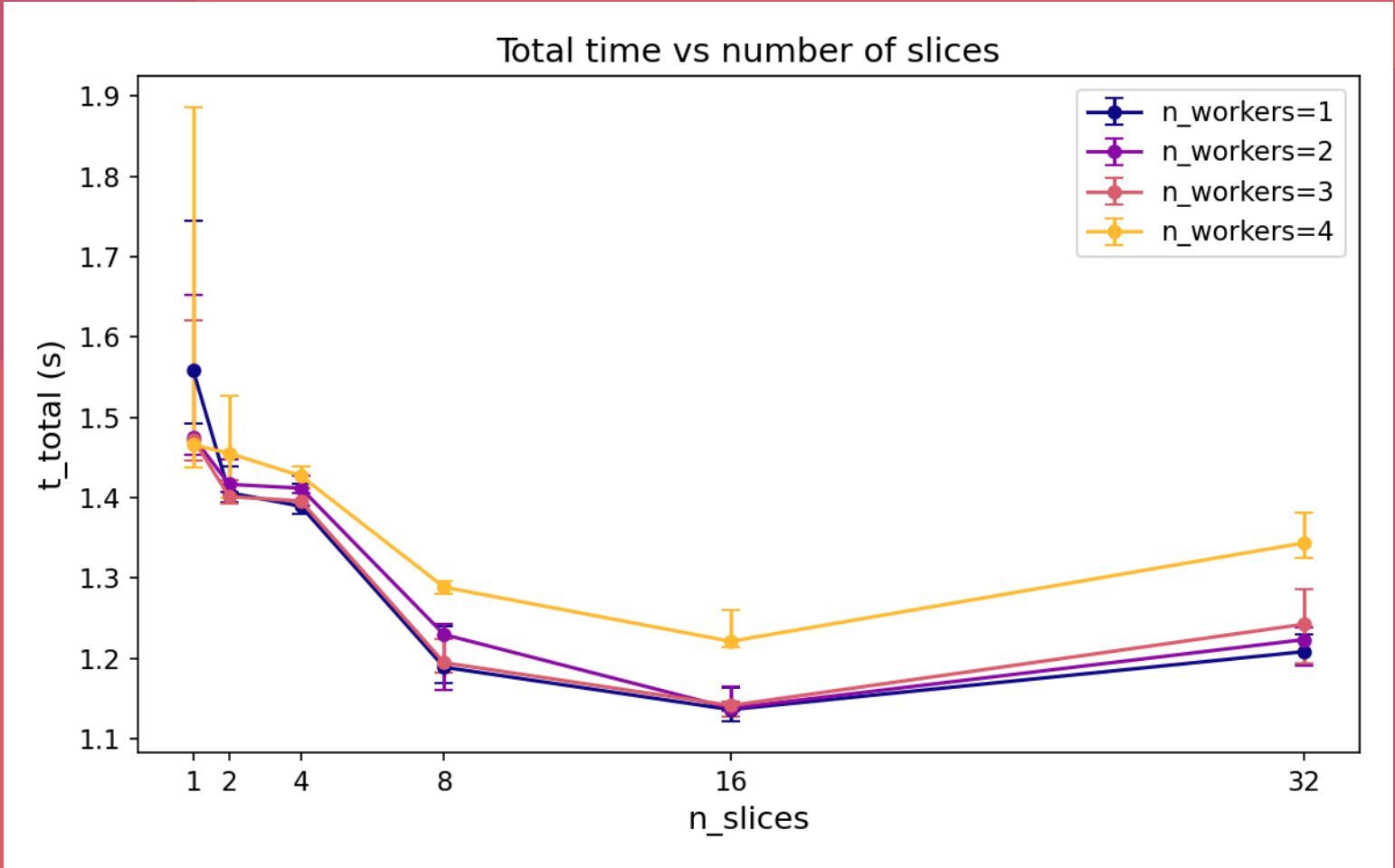
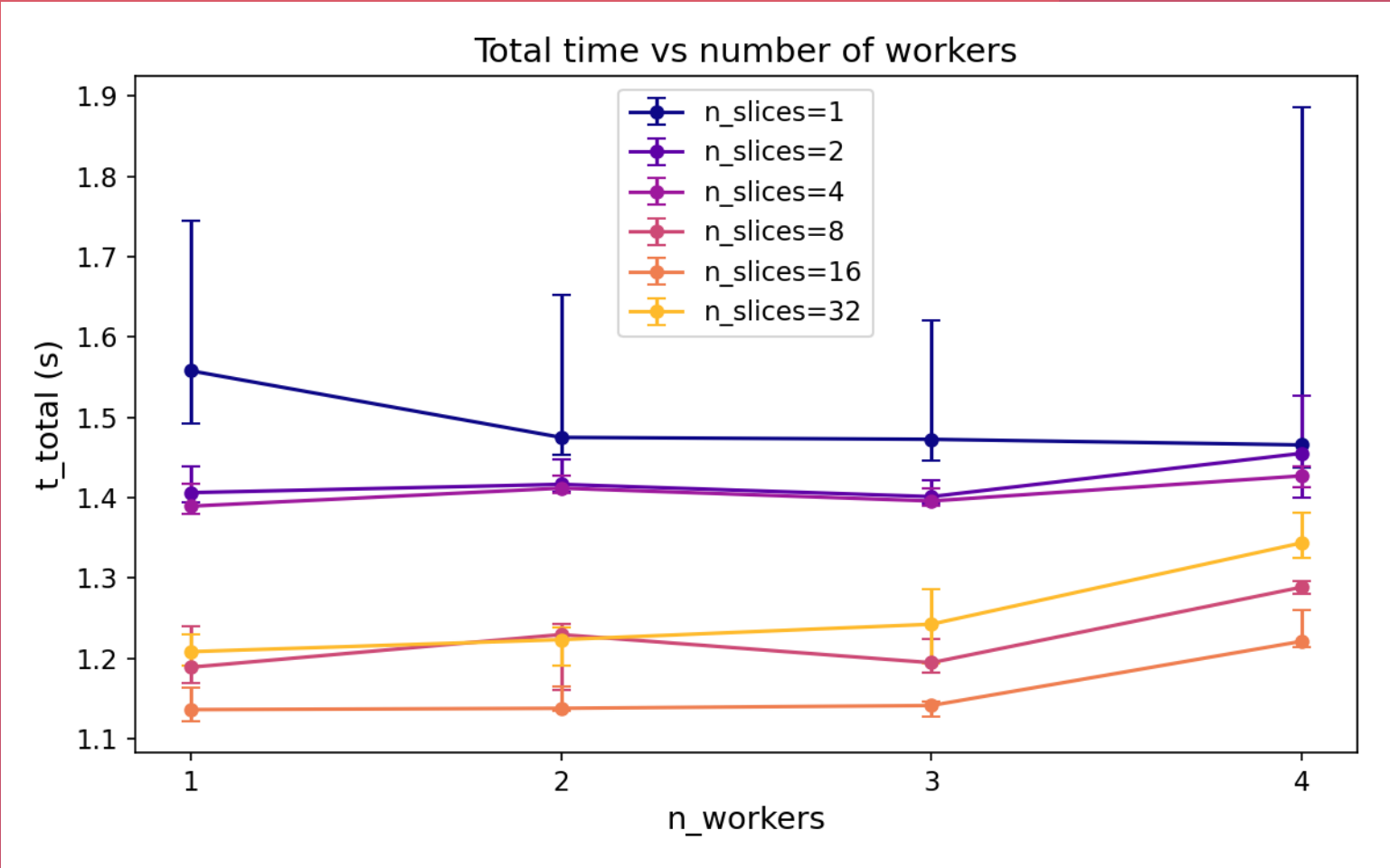
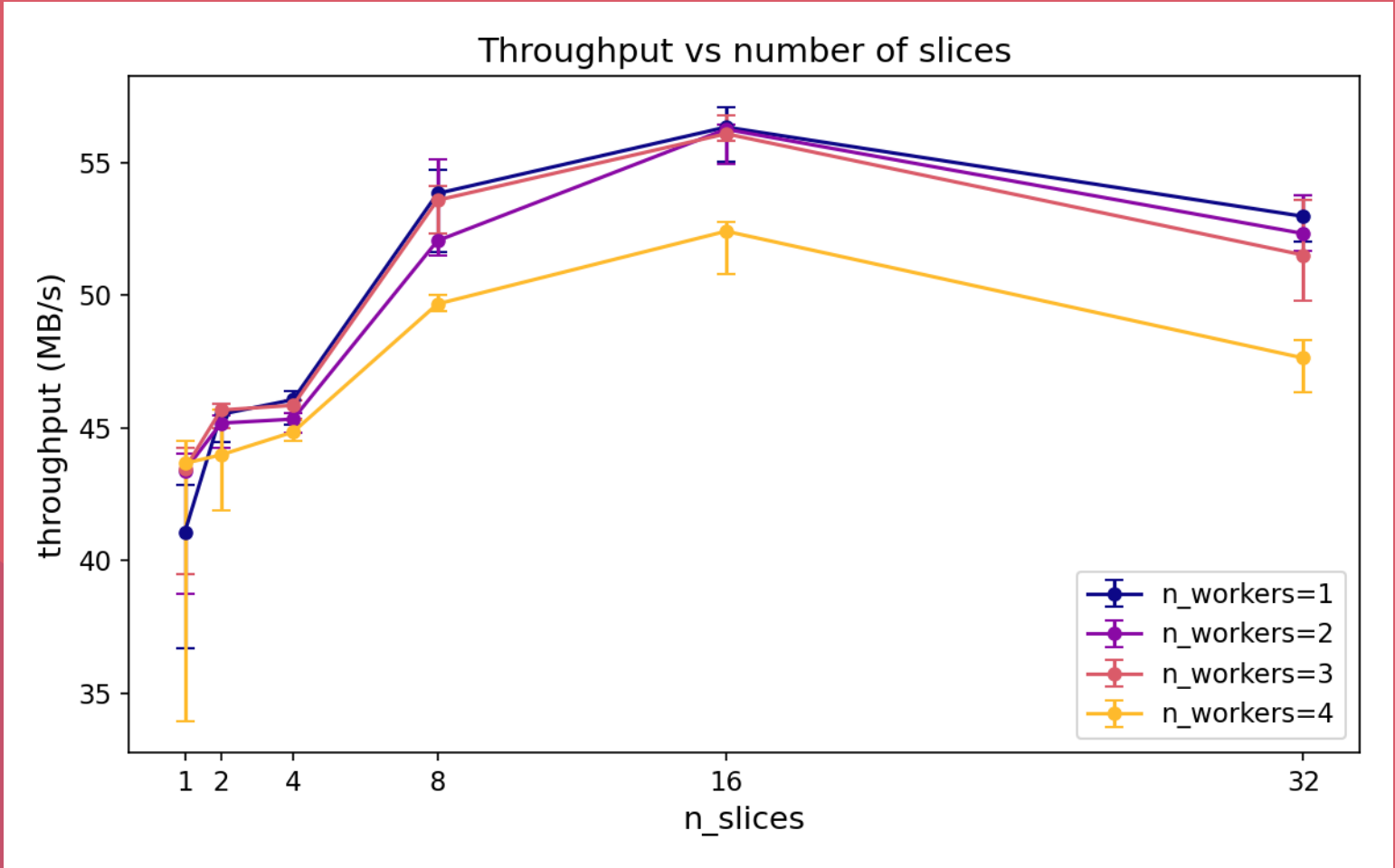
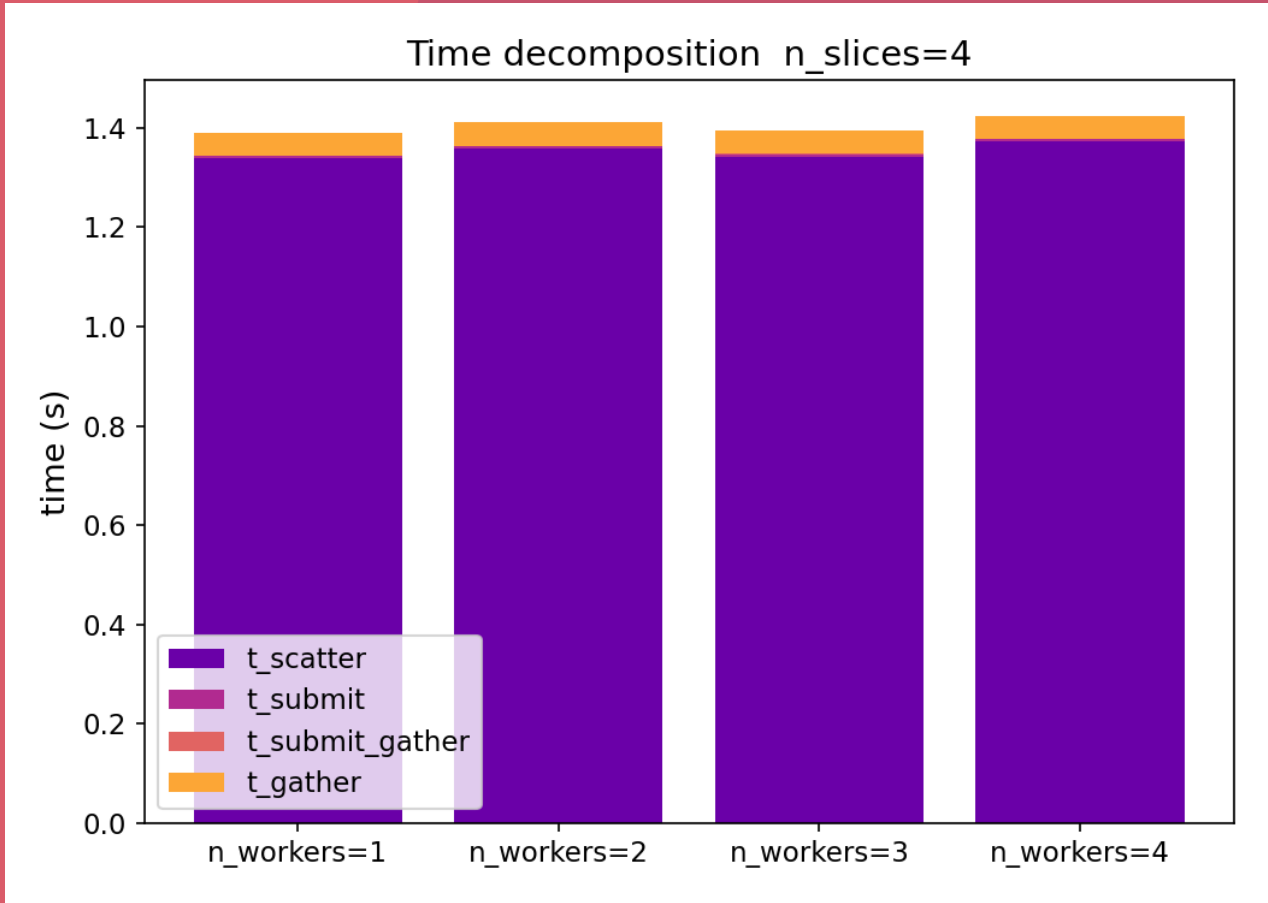


BANCHMARKS





BANCHMARKS (CLUSTER)



ac = as_completed()

```
with bench.timed(batch_id, "dask_submit"):
    # client.scatter() load data directly on a worker and returns a object that points to where data now live
    # it is faster w.r.t. passing directly the data when they are large (16MB in this case)
    1/2 raw_future = client.scatter(raw)
    chunk_future = client.submit(compute_fft_chunk, raw_future, header_len, header["n_bytes_i"], header["n_bytes_q"])

    # store current value with right index
    pending[batch_id].append(chunk_future)

    # TEST: check slices gathering
    # print(f"batch {batch_id}: gathered {len(pending[batch_id])}/{n_slices} slice")

    # extract values from dictionary and assemble data
    # delete batch_id entry once data have been extracted
    if len(pending[batch_id]) == n_slices:
        chunk_futures = pending.pop(batch_id)
        # submit to combine function
        future = client.submit(combine_chunks, batch_id, timestamp, chunk_futures)
        submit_times[future] = time.perf_counter()
        ac.add(future)
        4 print(f"Batch {batch_id} submitted ({n_slices} chunks)", flush=True)
        3
```

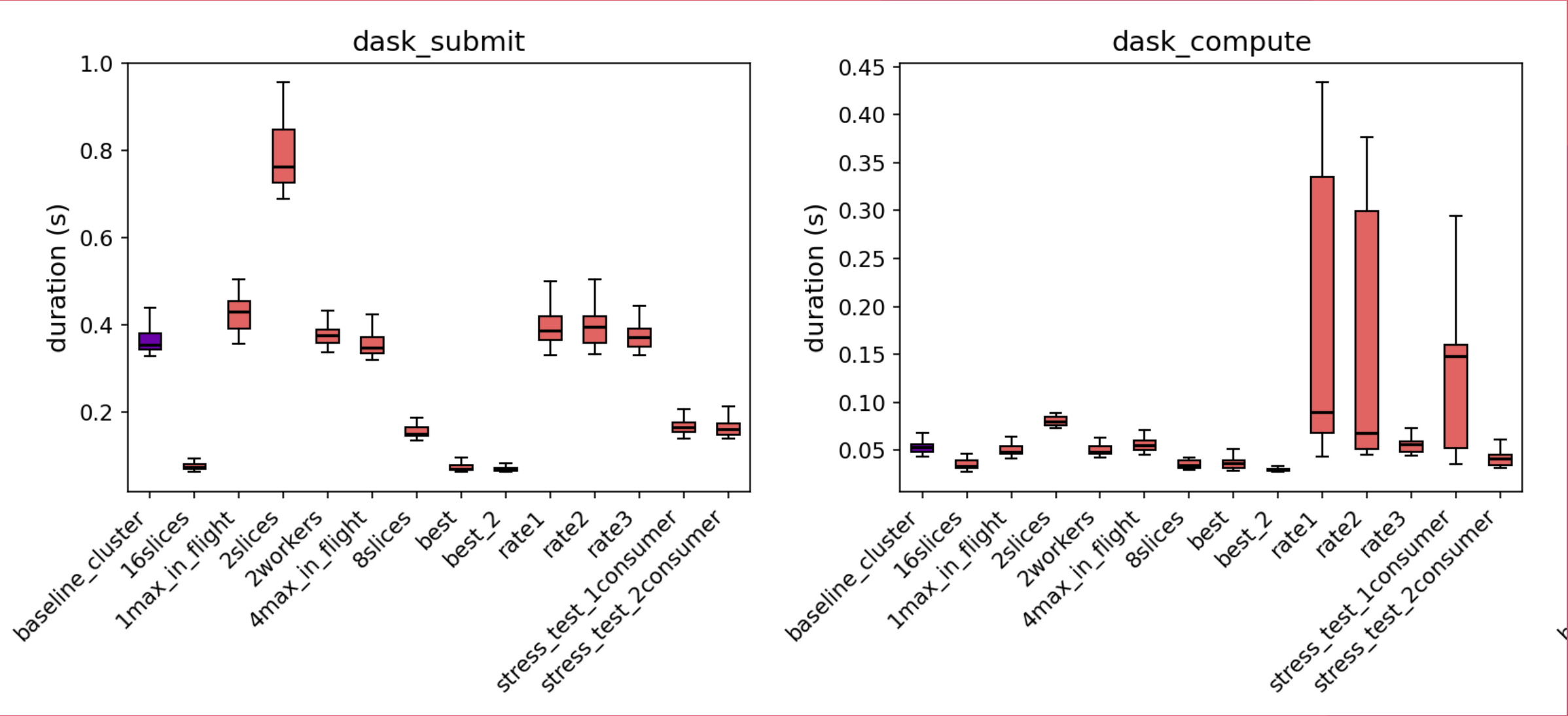
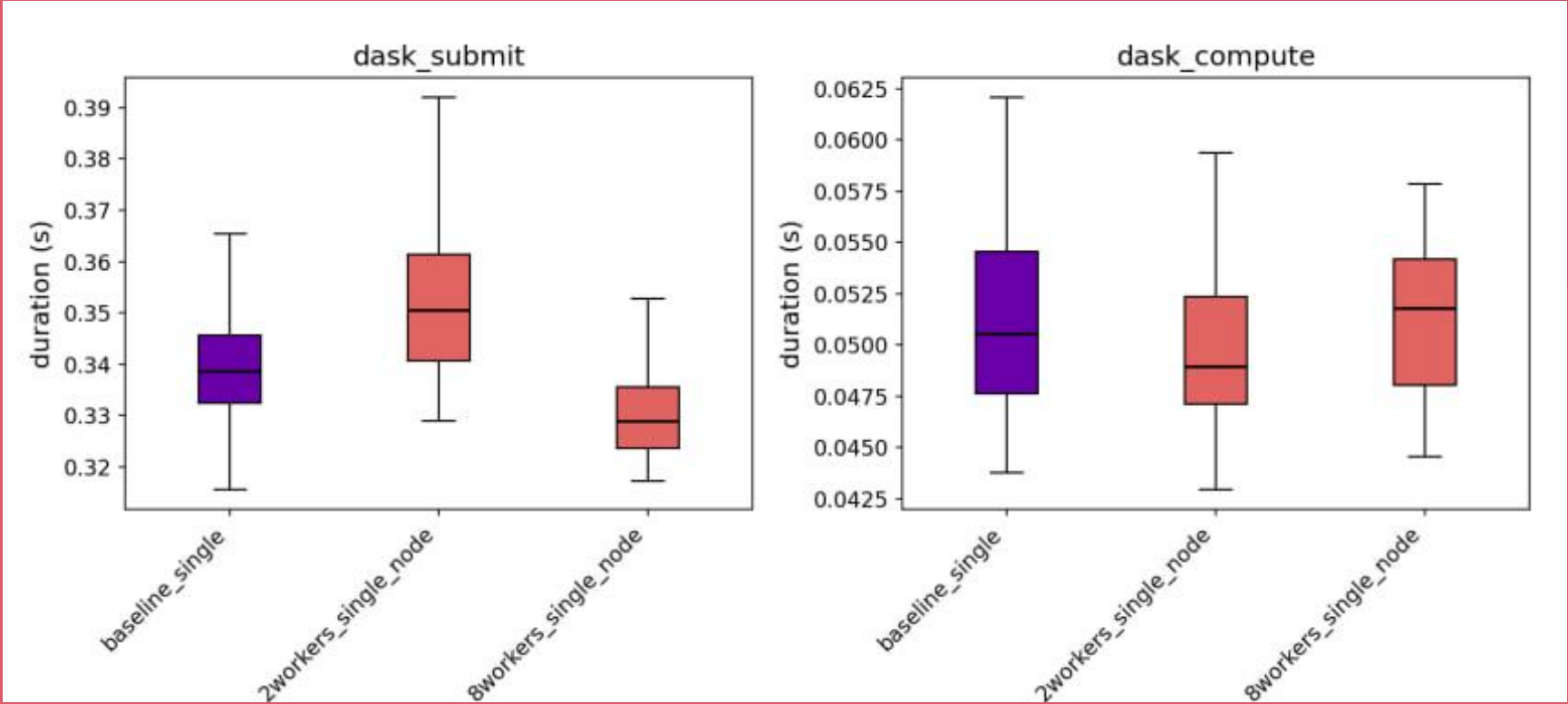
```
while True:
    try:
        # check if a future is ready otherwise "StopIteration"
        # using directly result = future.result() in a for loop would trigger "StopIteration" and end loop
        future = next(ac)
    except StopIteration:
        # wait for a brief moment if future is not ready
        time.sleep(0.02)
        continue

    # when future is retrieved the program can publish
    try:
        result = future.result()
```

ONLINE ON THE PIPELINE

What we implemented
differently on the pipeline

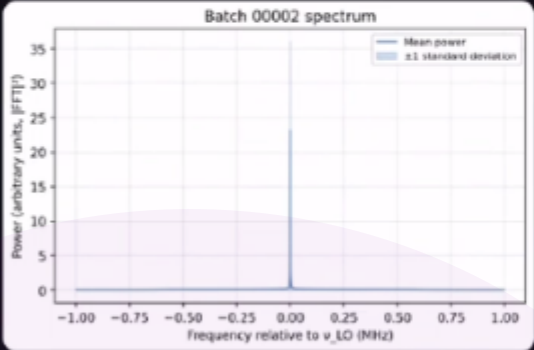
BANCHMARKS (CLUSTER)



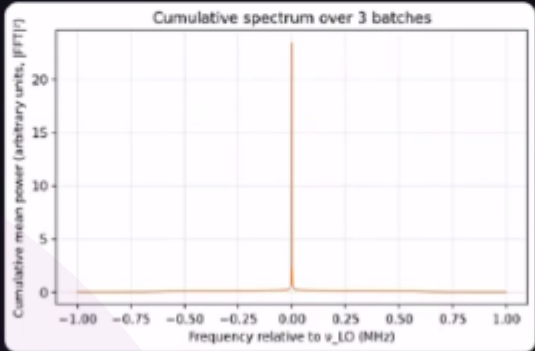
QUAX — Live spectrum monitor

Latest batch: 00002 — timestamp: 1789319271.8382123 — batches received so far: 3

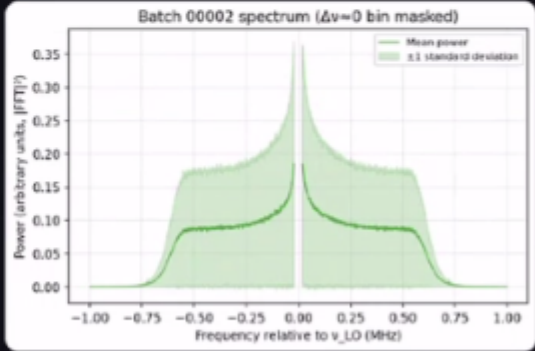
Latest batch spectrum



Cumulative spectrum (average over all batches)



Latest batch spectrum ($\Delta\nu \approx 0$ bin masked)



Dashboard

Shared state class:

```
class SharedState:
    def __init__(self):
        self.lock = threading.Lock()
        self.latest_batch_id = None
        self.latest_timestamp = None
        self.latest_mean = None
```

CONSUMER AND DASHBOARD

Two threads, one shared state:

One thread dedicated to Kafka, one to Streamlit (User Interface)

```
def snapshot(self):  
    with self.lock:  
        return (  
            self.latest_batch_id,  
            self.latest_timestamp,  
            None if self.latest_mean is None else self.latest_mean.copy(),
```

```
def update(self, batch_id, timestamp, mean, std):  
    with self.lock:  
        self.latest_batch_id = batch_id  
        self.latest_timestamp = timestamp  
        self.latest_mean = mean
```

Through the **lock** we
handle the two threads
application

CONSUMER

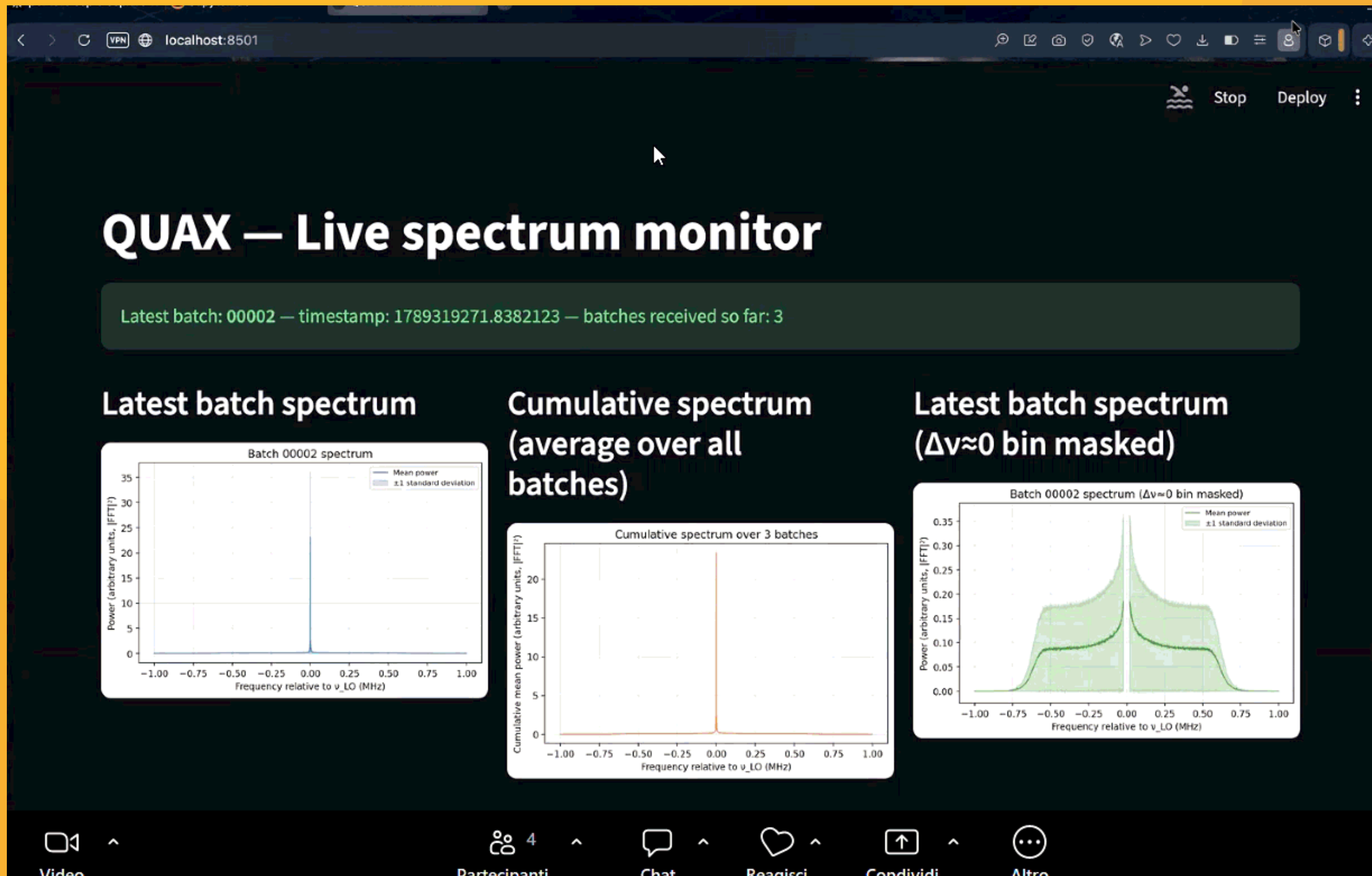
```
def consume_results(state: SharedState):
    consumer = KafkaConsumer(
        RESULTS_TOPIC,
        bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
        auto_offset_reset="earliest",
        value_deserializer=lambda v: json.loads(v.decode("utf-8")),
    )
    for message in consumer:
        msg = message.value
        batch_id = msg["batch_id"]
        timestamp = msg["timestamp"]
        mean = np.fft.fftshift(np.asarray(msg["mean"], dtype=np.float64))
        std = np.fft.fftshift(np.asarray(msg["std"], dtype=np.float64))
        state.update(batch_id, timestamp, mean, std)
```

The **decorator** is essential. If we call this function multiple times it won't declare a new state or start a new Kafka connection. It will simply return the initial object "state", which is being updated

```
@st.cache_resource
def get_state_and_start_thread():
    state = SharedState()
    thread = threading.Thread(target=consume_results, args=(state,), daemon=True)
    thread.start()
    return state
```

`Get_state_and_start_thread()`

**This function
instantiates the shared
state and starts the
thread.**



GROUP 06 MAPD-B 16.09.2026

GRAZIE PER L'ATTENZIONE



Erario Francesco, Lorenzetto Andrea, Mazzilli Laura, Zuccotti Nicola

